

DAX Functions

Complete Reference Guide

For Data Analysts using Microsoft Power BI

Aggregation | Logical | Text | Date & Time | Filter

Each function includes Definition • Syntax • Example • Output • Note

Introduction to DAX

- DAX stands for Data Analysis Expressions.
- It is the formula language used in Power BI, Power Pivot, and Analysis Services to create custom calculations, measures, and calculated columns.

DAX looks similar to Excel formulas but works very differently. Instead of calculating cell by cell, DAX works on entire columns and tables. And instead of static results, DAX measures recalculate dynamically based on the current filter context of the report — meaning the number changes instantly when a user clicks a slicer or applies a filter.

Two Ways DAX is Used

- **Measures**
 - Dynamic calculations evaluated at query time when the report renders.
 - A measure stores no data.
 - It computes fresh each time, responding to all active filters.
 - Measures are the preferred way to create aggregations and KPIs.
- **Calculated Columns**
 - A new column added to an existing table using a DAX formula.
 - Evaluated row by row at data refresh time and stored in the model.
 - Used for categorising rows or creating lookup values.

1. Aggregation Functions

Used to summarise data — totals, averages, counts, maximums, and minimums. These are the building blocks of every KPI measure.

SUM()

- Adds up all numeric values in a column and returns the total. The most commonly used DAX function. Every revenue or quantity KPI starts with SUM(). It only works on numeric columns and will not work on text.

Syntax:

| `SUM(<column>)`

Example:

| `Total Revenue = SUM(Sales[Price])`

Output: Adds all order Price across every row and returns total revenue, e.g. ₹1,85,430

AVERAGE()

Returns the arithmetic mean of all numeric values in a column — sum divided by count. AVERAGE() automatically ignores blank cells — it does not count them in the denominator. So if 10 rows exist but 2 are blank, it divides by 8, not 10.

Syntax:

```
| AVERAGE (<column>)
```

Example:

```
| Avg Delivery Time = AVERAGE(Sales[Delivery_Time])
```

Output: Sums all delivery times and divides by total count, e.g. 42 minutes

COUNT()

Counts the number of rows in a column that contain a numeric value. Ignores blank cells and text values. COUNT() only counts numbers. If your column contains text or blanks, use COUNTA() instead. For counting all rows in a table, always prefer COUNTROWS().

Syntax:

```
| COUNT (<column>)
```

Example:

```
| Count Ratings = COUNT(Sales[Rating])
```

Output: Returns the number of rows where Rating has a numeric value, e.g. 196 if 4 rows are blank

COUNTA()

Counts the number of non-blank values in a column. Works on text, numbers, and dates. Ignores only blank or null cells. Use COUNTA() when counting entries in a text column. COUNT() would return 0 for any text column. COUNTA() is the safer, more flexible choice for non-numeric columns.

Syntax:

```
| COUNTA (<column>)
```

Example:

```
| Count Customers = COUNTA(Sales[Customer_Name])
```

Output: Returns the number of rows where Customer_Name is not empty, e.g. 198

COUNTROWS()

Counts the total number of rows in a table regardless of data type or column content. This is the most reliable way to count orders, transactions, or any row-based metric. Unlike COUNT(), it does not depend on any specific column having a value — it simply counts rows.

Syntax:

```
| COUNTROWS (<table>)
```

Example:

```
| Total Orders = COUNTROWS(Sales)
```

Output: Returns 200 — the total number of rows in the Sales table

COUNTBLANK()

Counts the number of blank (empty or NULL) values in a column. Extremely useful for data quality checks. Use it to find out how many records are missing a key field before publishing your dashboard. A high blank count signals a data collection problem.

Syntax:

```
COUNTBLANK(<column>)
```

Example:

```
Missing Ratings = COUNTBLANK(Sales[Rating])
```

Output: Returns the number of rows where Rating has no value, e.g. 4

DISTINCTCOUNT()

Counts the number of unique (distinct) values in a column. Each unique value is counted only once, no matter how many times it repeats. Use this when you want to know how many unique customers placed orders, how many unique products were sold, or how many unique Showrooms were active — not the total transaction count.

Syntax:

```
DISTINCTCOUNT(<column>)
```

Example:

```
Active Showroom = DISTINCTCOUNT(Sales[Showroom_ID])
```

Output: Even if Showroom S001 appears 50 times in orders, it is counted as 1 — returns e.g. 28 unique Showroom

MAX()

Returns the largest value in a column. MAX() also works on date columns — MAX([Order_date]) returns the most recent date in the dataset. Commonly used to find the latest transaction date or highest delivery time.

Syntax:

```
MAX(<column>)
```

Example:

```
Highest Order = MAX(Sales[Price])
```

Output: Returns the single highest Price in the table, e.g. ₹1,195.50

MIN()

Returns the smallest value in a column. Like MAX(), MIN() also works on dates — MIN([Sales]) returns the earliest date in the dataset. Use together with MAX() to show the full data range in a KPI card.

Syntax:

```
MIN(<column>)
```

Example:

```
Lowest Order = MIN(Sales[Price])
```

Output: Returns the single lowest order Price, e.g. ₹82.00

MAXX()

Iterates over each row of a table, evaluates an expression per row, and returns the maximum result from all those row-level evaluations. Use MAXX() when the value you want the maximum of does not exist as a stored column — you need to calculate it row by row first and then find the max of those results.

Syntax:

```
| MAXX(<table>, <expression>)
```

Example:

```
| Max Revenue Per Item = MAXX(Sales, Sales[Price] * Sales[Quantity])
```

Output: Calculates Price x Quantity for every row, then returns the single highest value found, e.g. ₹3,600

MINX()

Iterates over each row of a table, evaluates an expression per row, and returns the minimum result. Same logic as MAXX() but returns the minimum. Useful for finding the cheapest calculated value in a derived expression rather than a stored column.

Syntax:

```
| MINX(<table>, <expression>)
```

Example:

```
| Min Revenue Per Item = MINX(Sales, Sales[Price] * Sales[Quantity])
```

Output: Calculates Amount x Quantity for each row, then returns the single lowest value found

SUMX()

Iterates over every row of a table, evaluates an expression for each row, and then sums all the row-level results together. SUMX() is needed when the value you want to sum must be calculated per row first — it does not exist as a ready-made column. If you used SUM() here, it would not work because SUM() cannot handle expressions, only columns.

Syntax:

```
| SUMX(<table>, <expression>)
```

Example:

```
| Weighted Revenue = SUMX(Sales, Sales[Price] * Sales[Quantity])
```

Output: Multiplies Amount x Quantity for every row individually, then adds all results together for total weighted revenue

AVERAGEX()

Iterates over every row of a table, evaluates an expression per row, and returns the average of all row-level results. This gives a true average of the per-row calculation — different from simply dividing total revenue by total quantity. The difference matters in financial and pricing analysis.

Syntax:

```
| AVERAGEX(<table>, <expression>)
```

Example:

```
| Avg Unit Price = AVERAGEX(Sales, Sales[Price] / Sales[Quantity])
```

Output: Calculates price per unit for every row first, then averages those unit prices across all orders, e.g. ₹185 per item

COUNTX()

Iterates over each row of a table, evaluates an expression per row, and counts how many results are not blank. COUNTX() is useful when you need to count rows based on a computed condition rather than a direct column value. In most simple cases COUNTROWS(FILTER(...)) achieves the same result more readably.

Syntax:

```
| COUNTX(<table>, <expression>)
```

Example:

```
| Count High Value Orders = COUNTX(FILTER(Sales, Sales[Price] > 500),  
| Sales[Order_id])
```

Output: Counts how many orders have an Price greater than ₹500, e.g. 63

2. Logical Functions

Used to test conditions and return different results based on whether conditions are true or false.

IF()

Tests a condition and returns one value if the condition is TRUE, and a different value if the condition is FALSE. The most fundamental logic function in DAX. Use it for any simple binary decision — pass/fail, above/below target, yes/no. For more than two outcomes, use SWITCH() instead of nesting multiple IFs.

Syntax:

```
| IF(<condition>, <value_if_true>, <value_if_false>)
```

Example:

```
| Order Status = IF(Sales[Price] > 500, "High Value", "Normal")
```

Output: Amount ₹850 returns High Value | Amount ₹200 returns Normal

Nested IF()

Multiple IF() functions placed inside each other to handle more than two possible outcomes. Nested IFs work but become very difficult to read and maintain beyond 3 levels. If you have more than 2 conditions, always prefer SWITCH(TRUE()) — it is cleaner, faster, and easier to update.

Syntax:

```
| IF(<condition1>, <result1>, IF(<condition2>, <result2>, <result_else>))
```

Example:

```
| Rating Label = IF([Rating] >= 4, "Positive", IF([Rating] = 3, "Neutral",  
| "Negative"))
```

Output: Rating 5 returns Positive | Rating 3 returns Neutral | Rating 1 returns Negative

SWITCH()

Evaluates a single expression against a list of exact values and returns the matching result. Works like a lookup table of fixed values. SWITCH() is perfect when you are matching against exact fixed values like category names, status codes, or payment modes. It is much cleaner than chaining multiple IF() functions.

Syntax:

```
| SWITCH(<expression>, <value1>, <result1>, <value2>, <result2>, ...,  
| <else_result>)
```

Example:

```
| Payment Label = SWITCH([Payment_Mode], "UPI", "Digital", "Card", "Digital",  
| "Cash", "Physical", "Unknown")
```

Output: UPI returns Digital | Cash returns Physical | anything else returns Unknown

SWITCH(TRUE())

A special and very powerful form of SWITCH used for range-based or complex conditions. Evaluates each condition in order and returns the result of the first one that is TRUE. The most recommended pattern for creating category columns based on numeric ranges. Conditions are evaluated top to bottom — the first TRUE wins. Always put the most restrictive condition first.

Syntax:

```
SWITCH(TRUE(), <condition1>, <result1>, <condition2>, <result2>, ..., <else_result>)
```

Example:

```
Revenue Category = SWITCH(TRUE(), [c] < 200, "Low", [Price] < 600, "Medium", "High")
```

Output: ₹150 returns Low | ₹400 returns Medium | ₹900 returns High

AND()

Returns TRUE only if both conditions are TRUE at the same time. If either condition is FALSE, the result is FALSE. In DAX you can also use the && operator instead of AND() — both do exactly the same thing. The && style is more commonly used by experienced DAX writers as it is shorter.

Syntax:

```
AND(<condition1>, <condition2>)
```

Example:

```
Premium Order = AND([Rating] >= 4, [Price] > 500)
```

Output: Rating 5 and Amount ₹700 returns TRUE | Rating 5 and Amount ₹200 returns FALSE

OR()

Returns TRUE if at least one of the two conditions is TRUE. Both conditions being FALSE is the only way to get a FALSE result. In DAX you can also use the || operator instead of OR(). For checking more than two values, use the IN operator: [Payment_Mode] IN {"UPI", "Card"} — cleaner for multiple options.

Syntax:

```
OR(<condition1>, <condition2>)
```

Example:

```
Digital Payment = OR([Payment_Mode] = "UPI", [Payment_Mode] = "Card")
```

Output: Payment UPI returns TRUE | Payment Card returns TRUE | Payment Cash returns FALSE

NOT()

Reverses a logical value — turns TRUE into FALSE and FALSE into TRUE. Use NOT() when it is easier to define what something is not rather than listing everything it could be. Especially useful when there are many possible values and you only want to exclude one specific value.

Syntax:

```
| NOT(<condition>)
```

Example:

```
| Not Delivered = NOT([Delivery_Status] = "Delivered")
```

Output: Status Delivered returns FALSE | Status Cancelled returns TRUE | Status Pending returns TRUE

IFERROR()

Evaluates an expression and returns a specified value if that expression produces an error. If no error occurs, it returns the expression result normally. Always wrap calculations that could produce errors — especially divisions, SEARCH(), FIND(), RELATED(), and VALUE() functions. Without IFERROR(), a single error in one row can break your entire visual.

Syntax:

```
| IFERROR(<expression>, <value_if_error>)
```

Example:

```
| Safe Avg = IFERROR(DIVIDE([Total Revenue], [Total Orders]), 0)
```

Output: If Total Orders = 0 which would cause a division error, returns 0 instead of an error message

ISBLANK()

Returns TRUE if a value is blank or NULL, and FALSE if the value contains any data. Commonly used inside IF() to handle missing data gracefully. Example: IF(ISBLANK([Rating]), "No Rating", [Rating]) shows No Rating instead of a blank in your visual.

Syntax:

```
| ISBLANK(<value>)
```

Example:

```
| Missing Name = ISBLANK(Sales[Customer_Name])
```

Output: Empty cell returns TRUE | Rahul Sharma returns FALSE

ISNUMBER()

Returns TRUE if a value is a number, and FALSE if it is text, blank, or any other data type. Use ISNUMBER() for data validation — detecting rows where a numeric column accidentally contains text. Often used together with IF() to flag bad data rows during data quality analysis.

Syntax:

```
| ISNUMBER(<value>)
```

Example:

```
| Is Valid Amount = ISNUMBER(Sales[Price])
```

Output: 850 returns TRUE | Eight Fifty returns FALSE | blank returns FALSE

ISTEXT()

Returns TRUE if a value is a text string, and FALSE if it is a number, date, blank, or any other type. Useful for detecting columns where numbers are stored as text — a common issue after importing CSVs. If ISTEXT() returns TRUE on a numeric column, you need to convert it using VALUE() before using SUM() or AVERAGE().

Syntax:

```
| ISTEXT(<value>)
```

Example:

```
| Is Text Check = ISTEXT(Sales[Delivery_Time])
```

Output: 45 min returns TRUE | 45 returns FALSE

IN Operator

Checks whether a value matches any value in a specified list. Returns TRUE if a match is found — a cleaner alternative to multiple OR() conditions. The IN operator is not a function but a DAX keyword used inside IF(), CALCULATE(), or FILTER(). It is the cleanest way to check membership in a list — far more readable than chaining multiple OR() conditions.

Syntax:

```
| <value> IN {<value1>, <value2>, <value3>}
```

Example:

```
| Digital Payment = [Payment_Mode] IN {"UPI", "Card"}
```

Output: UPI returns TRUE | Card returns TRUE | Cash returns FALSE

3. Text Functions

Used to clean, format, combine, and manipulate text values in columns and measures.

UPPER()

Converts all characters in a text string to UPPERCASE. Useful for standardising text before comparison — pune, Pune, and PUNE all become PUNE when passed through UPPER(). This makes inconsistent data entries behave as one single value.

Syntax:

| UPPER(<text>)

Example:

| UPPER([City])

Output: Pune returns PUNE

LOWER()

Converts all characters in a text string to lowercase. Use LOWER() when you want to compare two text values without worrying about capitalisation differences. Combining LOWER() on both sides of a comparison ensures a true case-insensitive match.

Syntax:

| LOWER(<text>)

Example:

| LOWER([City])

Output: Pune returns pune

TRIM()

Removes all leading spaces (before the text) and trailing spaces (after the text). Also reduces multiple spaces between words down to a single space. One of the most commonly used cleaning functions in any Power BI project. Always apply TRIM() to name, address, and category columns entered manually — extra spaces cause the same person or item to appear as two different entries in slicers and charts.

Syntax:

| TRIM(<text>)

Example:

| Clean Name = TRIM([Customer_Name])

Output: Rahul Sharma returns Rahul Sharma | Priya Patil returns Priya Patil

LEN()

Returns the total number of characters in a text string, including spaces, punctuation, and special characters.

Use LEN() for data validation — detecting suspiciously short entries, finding blank fields where LEN = 0, or checking if a code has the correct number of characters.

Syntax:

| LEN(<text>)

Example:

| Name Length = LEN([Customer_Name])

Output: Rahul Sharma returns 12 | Rahul Sharma returns 16 because spaces are counted

LEFT()

Extracts a specified number of characters from the beginning (left side) of a text string. Use LEFT() when a code, ID, or reference number has a meaningful prefix stored at the start — such as a country code, department code, or category prefix.

Syntax:

| LEFT(<text>, <num_chars>)

Example:

| ID Prefix = LEFT([Order_id], 3)

Output: ORD0042 returns ORD

RIGHT()

Extracts a specified number of characters from the end (right side) of a text string. Use RIGHT() to extract the numeric or sequential part of an ID stored at the end of a field. Also useful for extracting file extensions, country suffixes, or year digits stored at the end of a code.

Syntax:

| RIGHT(<text>, <num_chars>)

Example:

| Order Number = RIGHT([Order_id], 4)

Output: ORD0042 returns 0042

MID()

Extracts characters from the middle of a text string, starting at a specified position and taking a specified number of characters. Use MID() when the information you need is embedded in the middle of a structured string — such as extracting a year, region code, or batch number from a combined reference field. Position counting starts at 1, not 0.

Syntax:

```
| MID(<text>, <start_position>, <num_chars>)
```

Example:

```
| MID("ORD-2024-042", 5, 4)
```

Output: Starts at position 5, takes 4 characters and returns 2024

CONCATENATE()

Joins exactly two text strings into one combined string. CONCATENATE() only accepts two arguments. For joining three or more values, use the & operator instead — it is more flexible and widely preferred. Example: [Customer_Name] & " - " & [City] returns Rahul Sharma - Pune.

Syntax:

```
| CONCATENATE(<text1>, <text2>)
```

Example:

```
| CONCATENATE([Customer_Name], " - ")
```

Output: Rahul Sharma combined with - returns Rahul Sharma -

SUBSTITUTE()

Finds every occurrence of a specific text within a string and replaces it with new text. One of the most useful cleaning functions. Use it to strip unwanted suffixes like min, kg, INR, or units that are stored as part of a text field. After substitution, use VALUE() to convert the cleaned result into a number for calculations.

Syntax:

```
| SUBSTITUTE(<text>, <old_text>, <new_text>)
```

Example:

```
| Clean Time = SUBSTITUTE([Price], " INR", "")
```

Output: 4500 INR returns 4500

REPLACE()

Replaces a specific portion of a text string based on position and length — regardless of what the original characters are. REPLACE() works by position, not by value — it replaces whatever characters sit at that location. Use SUBSTITUTE() when you know what text to replace. Use REPLACE() when you know where (position) to replace.

Syntax:

```
| REPLACE(<old_text>, <start_position>, <num_chars>, <new_text>)
```

Example:

```
| REPLACE("ORD0042", 1, 3, "INV")
```

Output: Replaces first 3 characters with INV and returns INV0042

SEARCH()

Finds the starting position of one text string within another text string. Case-insensitive — uppercase and lowercase are treated the same. SEARCH() returns an error (not blank) when the text is not found. Always wrap it inside IFERROR() for safe use.

Example: IFERROR(SEARCH("Pune", [City]), 0) returns 0 instead of an error when not found.

Syntax:

```
| SEARCH(<find_text>, <within_text>)
```

Example:

```
| SEARCH("Pune", [City])
```

Output: Returns 1 if City starts with Pune | Returns an error if Pune is not found

FIND()

Same as SEARCH() — finds the starting position of one text within another. The key difference is that FIND() is case-sensitive. Use FIND() only when capitalisation matters in your search. In most real-world data cleaning scenarios, SEARCH() is preferred because inconsistent casing is common. Always wrap FIND() in IFERROR() as it errors when text is not found.

Syntax:

```
| FIND(<find_text>, <within_text>)
```

Example:

```
| FIND("pune", [City])
```

Output: Returns position if pune (lowercase) is found | Errors if Pune or PUNE is found instead

FORMAT()

Converts a number, date, or currency value into a formatted text string using a custom format pattern. FORMAT() always returns text — the result cannot be used in numeric calculations. Common format strings: dd/mm/yyyy for dates, ₹#,##0.00 for Indian currency, 0.00% for percentages, MMM YYYY for month-year chart labels.

Syntax:

```
| FORMAT(<value>, <format_string>)
```

Example:

```
| Month Label = FORMAT([Order_date], "MMM YYYY")
```

Output: 2024-01-15 returns Jan 2024 | 2024-03-20 returns Mar 2024

VALUE()

Converts a text string that looks like a number into an actual numeric value that can be used in calculations. Essential when numeric data is stored as text after importing CSVs or Excel files. Without VALUE(), functions like SUM(), AVERAGE(), and DIVIDE() will not work on that column. Always pair VALUE() with IFERROR() to handle rows that cannot be converted.

Syntax:

```
| VALUE(<text>)
```

Example:

```
| VALUE("450")
```

Output: 450 stored as text returns 450 as a usable number

TEXT()

Converts a number or date into a text string using a custom display format. The opposite of VALUE(). TEXT() is used when you need to display a formatted number as a text label — such as inside a concatenated string or a custom tooltip. Once converted to text with TEXT(), the value can no longer be used in numeric calculations.

Syntax:

```
| TEXT(<value>, <format_string>)
```

Example:

```
| Formatted Amount = TEXT([Price], "₹#,##0.00")
```

Output: 850.5 returns ₹850.50 | 1200 returns ₹1,200.00

BLANK()

Returns a blank value — the DAX equivalent of NULL or empty. : Returning BLANK() from a measure is better than returning 0 in many cases — Power BI automatically hides blank data points from line charts and bar charts, keeping visuals clean. Returning 0 would show a visible zero data point, which can be misleading.

Syntax:

```
| BLANK()
```

Example:

```
| Safe Value = IF([Price] = 0, BLANK(), [Price])
```

Output: Order Amount 0 returns blank (nothing displayed) | Any other value returns the value as-is

4. Date & Time Functions

Used to extract, calculate, and manipulate date and time values — essential for trend analysis and time intelligence.

TODAY()

Returns the current date. Updates automatically every time the report refreshes — no manual input needed. TODAY() returns only the date with no time component. It recalculates on every report refresh, so any measure using TODAY() is always current. Use it for calculating how old an order is, how many days since last purchase, or how current your data is.

Syntax:

| TODAY ()

Example:

| Days Since Order = DATEDIFF([Order_date], TODAY(), DAY)

Output: If order was placed on 01 Jan 2024 and today is 21 May 2026, returns 871 days

NOW()

Returns the current date and time together. Updates on every report refresh. : Use NOW() when you need to display an exact timestamp of when the report was last refreshed — commonly shown in a footer card on dashboards. The difference from TODAY() is that NOW() includes hours, minutes, and seconds.

Syntax:

| NOW ()

Example:

| Last Refreshed = NOW()

Output: Returns current date and time, e.g. 21/05/2026 14:32:00

DATE()

Creates a proper date value from three separate numbers — year, month, and day. Use DATE() when year, month, and day are stored in separate columns and need to be combined into one proper date field. Also useful for creating fixed reference dates in measures — such as a financial year start date for YTD calculations.

Syntax:

| DATE(<year>, <month>, <day>)

Example:

| Start Date = DATE(2024, 1, 1)

Output: Creates the date 01 January 2024

YEAR()

Extracts the 4-digit year number from a date value. Use YEAR() to create a year column for grouping, filtering, and year-level slicers. Essential for year-over-year comparison charts. Pair it with MONTH() to build a complete time hierarchy for drill-down visuals.

Syntax:

```
| YEAR(<date>)
```

Example:

```
| Order Year = YEAR([Order_date])
```

Output: 2024-03-15 returns 2024 | 2025-11-20 returns 2025

MONTH()

Extracts the month number (1 through 12) from a date value. Always create a Month Number column alongside a Month Name column. Month Name is used for display on chart axes, but Month Number is used to sort it correctly — otherwise Power BI sorts months alphabetically instead of chronologically.

Syntax:

```
| MONTH(<date>)
```

Example:

```
| Order Month Num = MONTH([Order_date])
```

Output: 2024-03-15 returns 3 | 2024-11-20 returns 11

DAY()

Extracts the day of the month (1 through 31) from a date value. Use DAY() to analyse day-of-month patterns — for example, do orders spike at the end of the month (salary day effect)? This kind of insight is only possible when day is extracted separately.

Syntax:

```
| DAY(<date>)
```

Example:

```
| Order Day = DAY([Order_date])
```

Output: 2024-03-15 returns 15 | 2024-01-31 returns 31

WEEKDAY()

Returns the day of the week as a number. The return type argument controls which day is treated as day 1. Return type 2 is the most commonly used setting — it makes Monday = 1 and Sunday = 7, which matches the standard business week. Use this to analyse weekly order patterns — are weekends busier than weekdays?

Syntax:

```
| WEEKDAY(<date>, <return_type>)
```

Example:

```
| Day of Week = WEEKDAY([Order_date], 2)
```

Output: Monday returns 1 | Tuesday returns 2 | Sunday returns 7

DATEDIFF()

Calculates the difference between two dates in a specified unit — Days, Months, Years, Hours, Minutes, or Seconds. : The interval options are DAY, MONTH, YEAR, HOUR, MINUTE, and SECOND. Use DATEDIFF() for calculating order age, customer tenure, days since last purchase, or time between events. The result is always a whole number.

Syntax:

```
| DATEDIFF(<start_date>, <end_date>, <interval>)
```

Example:

```
| Order Age Days = DATEDIFF([Order_date], TODAY(), DAY)
```

Output: Order placed 30 days ago returns 30 | Order placed 1 year ago returns 365

EOMONTH()

Returns the last day of the month, a specified number of months before or after a start date. Use 0 to get the last day of the same month. Use 1 to get the last day of the next month. Widely used in financial reporting for period-end calculations and monthly close reports.

Syntax:

```
| EOMONTH(<start_date>, <months>)
```

Example:

```
| Month End = EOMONTH([Order_date], 0)
```

Output: Any date in January 2024 returns 31/01/2024 | Any date in February 2024 returns 29/02/2024

CALENDAR()

Creates a table of consecutive dates between a specified start date and end date — one row per date. CALENDAR() is used to create a dedicated Date/Calendar table — the recommended best practice in every Power BI project. A proper Date table enables all time intelligence functions to work correctly.

Syntax:

```
| CALENDAR(<start_date>, <end_date>)
```

Example:

```
| Date Table = CALENDAR(DATE(2023,1,1), DATE(2025,12,31))
```

Output: Creates a table with one row for every date from 01 Jan 2023 to 31 Dec 2025, giving 1,096 rows

CALENDARAUTO()

Automatically creates a date table covering the full date range found across all date columns in your entire data model — no manual start or end date needed. Easier and faster than CALENDAR() for most projects. Always mark the resulting table as a Date Table in Power BI: right-click table and select Mark as Date Table.

Syntax:

```
| CALENDARAUTO()
```

Example:

```
| Date Table = CALENDARAUTO()
```

Output: Scans all date columns in the model and creates a complete date table from the earliest to latest date found automatically

DATEVALUE()

Converts a date stored as text into a proper date value that Power BI can use for calculations and time intelligence. Essential when dates are imported as text strings — a very common issue with CSV files. Without converting to a proper date, time intelligence functions like DATESMTD() and DATESYTD() will not work.

Syntax:

```
| DATEVALUE(<date_text>)
```

Example:

```
| DATEVALUE("15 January 2024")
```

Output: 15 January 2024 stored as text returns 15/01/2024 as a proper date value

5. Filter Functions

The most powerful category in DAX. Used to control and manipulate filter context. CALCULATE is the king of all DAX functions.

CALCULATE()

The single most powerful and most used function in all of DAX. Evaluates any expression while modifying or replacing the current filter context with one or more new filters. CALCULATE() is the engine behind almost every advanced DAX measure. Without it, you cannot override filters, create segment comparisons, or build time intelligence measures. Master this one function and you will be able to solve 80% of all DAX challenges.

Syntax:

```
CALCULATE(<expression>, <filter1>, <filter2>, ...)
```

Example:

```
Delivered Revenue = CALCULATE([Total Revenue], Sales[Delivery_Status] = "Delivered")
```

Output: Calculates Total Revenue but only for rows where Delivery_Status equals Delivered, e.g. ₹1,42,500

CALCULATETABLE()

Works exactly like CALCULATE() but instead of returning a scalar value (single number), it returns an entire filtered table. CALCULATETABLE() is used when you need a filtered version of a table to pass into another function — such as COUNTROWS(), SUMX(), or AVERAGEX(). It is the table-returning equivalent of CALCULATE().

Syntax:

```
CALCULATETABLE(<table>, <filter1>, <filter2>, ...)
```

Example:

```
Delivered Orders Table = CALCULATETABLE(Sales, Sales[Delivery_Status] = "Delivered")
```

Output: Returns a virtual table containing only the rows where Delivery_Status equals Delivered

FILTER()

Returns a filtered version of a table — keeps only the rows that satisfy a specified condition. Always returns a table, never a single value. FILTER() is almost always used inside other functions like CALCULATE(), COUNTROWS(), SUMX(), or AVERAGEX() — not on its own. Use FILTER() when your condition involves multiple columns or complex logic that CALCULATE() alone cannot express.

Syntax:

```
FILTER(<table>, <condition>)
```

Example:

```
High Value Orders = FILTER(Sales, Sales[Price] > 500)
```

Output: Returns a virtual table of only the rows where Price is greater than ₹500

ALL()

Removes all filters from a specified table or column — completely ignores any active slicers, page filters, or visual filters. Returns all rows as if no filter exists. ALL() is essential for percentage-of-total calculations. Without ALL() in the denominator, both numerator and denominator would be filtered the same way — always returning 100%.

Syntax:

```
| ALL(<table_or_column>)
```

Example:

```
| Revenue % of Total = DIVIDE([Total Revenue], CALCULATE([Total Revenue],  
| ALL(Sales)), 0)
```

Output: If Pune revenue is ₹45,000 and grand total is ₹2,50,000, returns 0.18 which is 18%

ALLEXCEPT()

Removes all filters from a table except for the columns you specify. The specified columns keep their active filters — everything else is cleared. Use ALLEXCEPT() when you want to calculate a subtotal within one specific dimension while ignoring all other active filters. A common use case is calculating the percentage share of a Sales within its city.

Syntax:

```
| ALLEXCEPT(<table>, <column1>, <column2>, ...)
```

Example:

```
| City Total Revenue = CALCULATE([Total Revenue], ALLEXCEPT(Sales, Sales[City]))
```

Output: Removes all filters except City — calculates total revenue per city regardless of any other slicer selections on the page

ALLSELECTED()

Removes filters applied within the current visual but keeps filters applied by slicers and other external filters on the page. Gives you the total within the current slicer selection. ALL() ignores everything including slicers and gives the absolute grand total. ALLSELECTED() respects slicer selections but removes visual-level filters. Use ALLSELECTED() when you want percentages to always add up to 100% within the slicer selection.

Syntax:

```
| ALLSELECTED(<table_or_column>)
```

Example:

```
| Revenue % of Slicer Total = DIVIDE([Total Revenue], CALCULATE([Total Revenue],  
| ALLSELECTED(Sales)), 0)
```

Output: If a slicer filters to Mumbai only, the denominator becomes total Mumbai revenue, not grand total — each bar shows percentage within Mumbai

RELATED()

Fetches a single value from a related table by following the active relationship. Works row by row in calculated columns — like a VLOOKUP in Excel. RELATED() only works in the many-side table (fact table) looking up into the one-side (dimension table). It requires an active relationship between the two tables. Use it in calculated columns — not in measures.

Syntax:

```
| RELATED(<column>)
```

Example:

```
| Cuisine = RELATED(Showrooms[Cuisine])
```

Output: For each row in Sales, looks up and returns the of the Model corresponding Showroom — SUV, Sidan, Mini, etc.

RELATEDTABLE()

Returns all the rows from a related table that match the current row. The reverse direction of RELATED() — used from the dimension table side looking into the fact table. From a Showroom row, RELATEDTABLE(Sales) gives you all orders for that Showroom. Always wrap it inside an aggregation function like COUNTROWS(), SUMX(), or AVERAGEX() — it returns a table, not a single value.

Syntax:

```
| RELATEDTABLE(<table>)
```

Example:

```
| Order Count = COUNTROWS(RELATEDTABLE(Sales))
```

Output: Used in the Showrooms table — returns the number of orders placed at each Showroom, e.g. R001 has 12 orders

USERELATIONSHIP()

Activates a specific inactive relationship between two tables for the duration of a single CALCULATE() expression. Only one relationship between two tables can be active at a time — others are inactive. Used in role-playing dimension scenarios — where one fact table has multiple date columns (Order Date, Delivery Date, Payment Date) all connecting to the same Date table. USERELATIONSHIP() activates the correct one on demand.

Syntax:

```
| USERELATIONSHIP(<column1>, <column2>)
```

Example:

```
| Orders by Delivery Date = CALCULATE([Total Orders],  
| USERELATIONSHIP(Sales[Delivery_Date], Dates[Date]))
```

Output: Counts orders using Delivery_Date to join the Date table instead of the default Order_date relationship

KEEPFILTERS()

Adds a filter inside CALCULATE() without replacing or overriding the existing filter context — instead it intersects (combines) the new filter with the existing one. By default CALCULATE() replaces existing filters with the new filter. KEEPFILTERS() keeps the existing context filter and adds the new one on top. Use it when you want both conditions to apply simultaneously.

Syntax:

```
| KEEPFILTERS(<filter_expression>)
```

Example:

```
| Delivered in Mumbai = CALCULATE([Total Orders], KEEPFILTERS(Sales[City] =  
| "Mumbai"), Sales[Delivery_Status] = "Delivered")
```

Output: Returns orders that are BOTH in Mumbai AND Delivered — both filters apply together

REMOVEFILTERS()

Clears all filters from a specified table or column inside a CALCULATE() expression. Introduced as a cleaner and more readable alternative to ALL() when used inside CALCULATE(). REMOVEFILTERS() and ALL() produce the same result when used inside CALCULATE(). REMOVEFILTERS() is preferred in modern DAX writing because its name clearly communicates intent — you are removing filters.

Syntax:

```
| REMOVEFILTERS(<table_or_column>)
```

Example:

```
| Grand Total Revenue = CALCULATE([Total Revenue], REMOVEFILTERS(Sales))
```

Output: Returns total revenue with all filters removed — same as using ALL(Sales) inside CALCULATE()

HASONEVALUE()

Returns TRUE if a specified column has been filtered down to exactly one unique value in the current filter context. Returns FALSE if multiple values or no values are selected. Used to build dynamic titles, labels, and conditional measures that respond intelligently to slicer selections. If your measure requires exactly one value to make sense, wrap it in HASONEVALUE() to prevent incorrect results.

Syntax:

```
| HASONEVALUE(<column>)
```

Example:

```
| City Label = IF(HASONEVALUE(Sales[City]), VALUES(Sales[City]), "Multiple  
| Cities")
```

Output: One city selected in slicer shows city name e.g. Pune | Multiple cities selected shows Multiple Cities

SELECTEDVALUE()

Returns the value of a column when exactly one value is in the filter context. Returns an alternate result if zero or multiple values exist. A cleaner shortcut for the IF(HASONEVALUE(), VALUES(), else) pattern. SELECTEDVALUE() is the modern, cleaner replacement for the IF(HASONEVALUE(col), VALUES(col), else) pattern. Use it for dynamic report titles, KPI card subtitles, and conditional labels that respond to slicer selections.

Syntax:

```
| SELECTEDVALUE(<column>, <alternate_result>)
```

Example:

```
| City Title = SELECTEDVALUE(Sales[City], "All Cities")
```

Output: One city selected returns Pune | No selection or multiple returns All Cities

ISFILTERED()

Returns TRUE if any direct filter is currently applied to the specified column — regardless of how many values are filtered to. Use ISFILTERED() to conditionally show or hide information based on whether a user has made a slicer selection. Helpful for building smart measures that behave differently when a filter is active.

Syntax:

```
| ISFILTERED(<column>)
```

Example:

```
| Filter Status = IF(ISFILTERED(Sales[City]), "City Filter Active", "Showing All Cities")
```

Output: Any city slicer selection active returns City Filter Active | No slicer selection returns Showing All Cities

VALUES()

Returns a single-column table of unique values from a column, respecting the current filter context. Also commonly used to retrieve a single value when a column is filtered to exactly one entry. : VALUES() is one of the most versatile functions in DAX — it works both as a table function (returning all unique values) and as a scalar retrieval tool when combined with HASONEVALUE(). It respects filter context fully.

Syntax:

```
| VALUES(<column>)
```

Example:

```
| Selected City = IF(HASONEVALUE(Sales[City]), VALUES(Sales[City]), "Multiple")
```

Output: When exactly one city is selected returns that city name e.g. Pune | Multiple selected returns Multiple

DISTINCT()

Returns a single-column table of unique values from a column, similar to VALUES() but ignores blank rows added by relationship mismatches. The practical difference between DISTINCT() and VALUES() is that VALUES() includes a blank row when there are unmatched keys in a relationship, while DISTINCT() does not. Use DISTINCT() for clean unique value lists.

Syntax:

| `DISTINCT(<column>)`

Example:

| `Unique Model = DISTINCT(Sales[])`

Output: Returns each unique model exactly once — X1, XE, IX, i4, Z4...